How Large Language Models Work

1. What Is a Large Language Model

A Large Language Model (LLM) is a type of neural network trained on massive amounts of text data to predict the next word, or more precisely the next token, in a sequence. Despite this relatively simple training objective, LLMs learn to capture grammar, facts, reasoning patterns, and even some limited problem-solving ability purely from statistical patterns in the training data.

Modern LLMs are built on the Transformer architecture, introduced in the 2017 paper 'Attention Is All You Need'. The Transformer replaced earlier recurrent architectures like RNNs and LSTMs with a mechanism called self-attention, which allows the model to weigh the relevance of every other token in a sequence when processing a given token, regardless of distance between them.

2. Tokenization

Before text can be processed by an LLM, it must be converted into tokens, which are numerical representations of subword units. Tokenization algorithms such as Byte Pair Encoding (BPE) break text into frequently occurring chunks, which may be whole words, parts of words, or even individual characters for rare terms.

This subword approach allows the model to handle a fixed-size vocabulary while still representing rare or unseen words by combining smaller known subword units, avoiding the need for an infinitely large vocabulary.

3. Embeddings and Self-Attention

Each token is converted into a dense numerical vector called an embedding, which captures semantic meaning in a high-dimensional space. Words with similar meanings tend to have embeddings that are close together in this space.

The self-attention mechanism then computes, for each token, a weighted combination of all other tokens in the input, using three learned projections called Query, Key, and Value vectors. This allows the model to determine which parts of the input are most relevant to understanding or generating each token.

Transformers use multiple attention heads in parallel, a technique called multi-head attention, allowing the model to attend to different types of relationships simultaneously, such as syntactic structure in one head and long-range topical relevance in another.

4. Training Process

LLMs are typically trained in multiple stages. The first stage, pretraining, involves training the model on a very large, diverse corpus of text using the next-token prediction objective. This stage requires enormous computational resources and is where the model acquires most of its general knowledge and language ability.

The second stage, often called fine-tuning or instruction tuning, involves further training the model on smaller, curated datasets of instructions and desired responses, teaching the model to follow user instructions more reliably.

A third stage, commonly Reinforcement Learning from Human Feedback (RLHF), uses human preference data to further align model outputs with what humans consider helpful, honest, and harmless, by training a reward model and using it to fine-tune the LLM's behavior.

5. Inference and Generation

During inference, the model generates text one token at a time. At each step, it produces a probability distribution over the entire vocabulary for the next token, and a sampling strategy, such as greedy decoding, top-k sampling, or nucleus (top-p) sampling, is used to select the next token.

Parameters like temperature control the randomness of this selection: lower temperature values make the output more deterministic and focused, while higher values increase diversity and creativity at the cost of coherence.

6. Limitations

LLMs are known to hallucinate, meaning they can generate plausible-sounding but factually incorrect information, since their knowledge is derived from statistical patterns rather than a verified, structured database of facts. This is a key motivation behind Retrieval-Augmented Generation (RAG) systems, which ground model outputs in retrieved, verifiable source documents rather than relying solely on the model's internal parametric knowledge.

Additionally, LLMs have a fixed context window, meaning there is a maximum number of tokens they can process in a single request, and their knowledge is limited to what was present in their training data up to a specific cutoff date.